

1. xTaskCreate()

2. Use Case (The "Why")

`xTaskCreate()` is the function used to **create a new task** and add it to the list of tasks managed by the FreeRTOS scheduler. It's the essential first step to introduce concurrency and multitasking into your application.

- **Analogy:** You are **hiring a new worker** (the task) on the job site (the RTOS), giving them a job description (the function), a dedicated workspace (the stack), and a priority level.
-

3. Syntax

C

```
BaseType_t xTaskCreate(  
    TaskFunction_t pvTaskCode,  
    const char * const pcName,  
    configSTACK_DEPTH_TYPE usStackDepth,  
    void * const pvParameters,  
    UBaseType_t uxPriority,  
    TaskHandle_t * const pxCreatedTask  
);
```

4. Parameters

Parameter	Type	Description
<code>pvTaskCode</code>	<code>TaskFunction_t</code>	A pointer to the function that contains the task's code. This function must never return (usually by having an infinite loop inside).
<code>pcName</code>	<code>const char * const</code>	A descriptive string name for the task (e.g., <code>"LED_Task"</code>). Used primarily for debugging and monitoring.

usStackDepth	<code>configSTACK_DEPTH_TYPE</code>	The stack size allocated for the task. Crucially, on ESP32/ESP-IDF, this is specified in 4-byte words, not bytes.
pvParameters	<code>void * const</code>	A pointer to data that will be passed into the task function. If no parameters are needed, pass <code>NULL</code> .
uxPriority	<code>UBaseType_t</code>	The priority at which the task should run. Higher numbers represent higher priority. The maximum is <code>configMAX_PRIORITIES - 1</code> .
pxCreatedTask	<code>TaskHandle_t *</code> <code>const</code>	An output pointer. If the task is created successfully, a unique handle (ID) for the task is written here. Pass <code>NULL</code> if the handle isn't needed.

Return Value:

- **pdPASS**: The task was created successfully.
- **errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY**: The system could not find enough heap memory to allocate the task's stack and Task Control Block (TCB).

5. Example

Here is how you would create a task named "MyTask" with a priority of 5, passing a simple integer as a parameter, and getting its handle:

```
// 1. Define the task's handle variable
TaskHandle_t xMyTaskHandle = NULL;

// 2. Define the task function
void vMyTask(void *pvParameters) {
    // Cast the parameter pointer back to an integer
    int task_id = (int)pvParameters;
    printf("Task ID: %d started.\n", task_id);

    // Tasks must contain an infinite loop
    for (;;) {
        // Do the task's work
        printf("Task %d running...\n", task_id);
        vTaskDelay(pdMS_TO_TICKS(1000)); // Delay for 1 second
    }
}
```

```

    }
}

// 3. Call xTaskCreate() from your main setup function
void app_main(void) {

    // Create the task
    BaseType_t xReturned = xTaskCreate(
        vMyTask,           // Task function code
        "MyTaskName",      // Task name (for debugging)
        2048,              // Stack size (in words, e.g., 2048 words * 4 bytes/word = 8KB)
        (void *)1,         // Parameter to pass (the ID '1')
        5,                 // Priority (5 is higher than default)
        &xMyTaskHandle      // Task handle output
    );

    if (xReturned == pdPASS) {
        printf("Task created successfully. Handle: %p\n", (void *)xMyTaskHandle);
    } else {
        printf("Task creation failed!\n");
    }
}

```

6. Workflow (What Happens Internally)

1. **Memory Allocation:** FreeRTOS uses the heap memory to allocate two things:
 - The **Task Control Block (TCB)**, which stores all the task's internal information (state, priority, stack pointers, etc.).
 - The **Task Stack**, a contiguous block of RAM of the size specified by `usStackDepth`.
2. **Initialization:** The task's stack is initialized. The **initial CPU context** (register values, program counter) is set up on the stack so that when the scheduler eventually runs this task, the CPU will start executing at the beginning of the `pvTaskCode` function, and the `pvParameters` will be passed in.
3. **State Change:** The newly created task is placed into the **Ready List** of the scheduler. It is now ready to run.
4. **Scheduler Check:** Since a new task has been created, FreeRTOS checks if the newly created task's priority (`uxPriority`) is higher than the currently running task's priority.
5. **Context Switch (if necessary):** If the new task has a higher priority, an immediate **context switch** occurs, and the new task begins execution immediately. If not, the

currently running task continues until it enters the Blocked state (e.g., via `vTaskDelay()`) or its time slice is up (preemption).

1. `xTaskCreatePinnedToCore()`

2. Use Case (The "Why")

This function is used to **create a new task and explicitly assign ("pin") it to run on a specific CPU core** (either Core 0 or Core 1) of the ESP32.

- **Why pin a task?** To leverage the dual-core architecture, ensuring time-critical tasks get exclusive use of a core, or to comply with specific peripheral driver requirements.
-

3. Syntax

C

```
BaseType_t xTaskCreatePinnedToCore(  
    TaskFunction_t pvTaskCode,  
    const char * const pcName,  
    uint32_t usStackDepth,  
    void * const pvParameters,  
    UBaseType_t uxPriority,  
    TaskHandle_t * const pxCreatedTask,  
    const BaseType_t xCoreID  
);
```

4. Parameters

Parameter	Type	Description
<code>pvTaskCode</code>	<code>TaskFunction_t</code>	Pointer to the task function.
<code>pcName</code>	<code>const char *</code> <code>const</code>	String name for the task.

usStackDepth	uint32_t	Stack size for the task (in 4-byte words on ESP32).
pvParameters	void * const	Pointer to data passed into the task function.
uxPriority	UBaseType_t	The priority of the task.
pxCreatedTask	TaskHandle_t *	Output pointer for the task handle.
xCoreID	const BaseType_t	The Core ID to run the task on. 0 (Core 0), 1 (Core 1), or tskNO_AFFINITY .

Export to Sheets

5. Return Value

The function returns a value of type **BaseType_t** (which is essentially an integer).

Return Value	Description
pdPASS	The task was created successfully and has been placed in the appropriate core's Ready List.
errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY	The system ran out of heap memory and could not allocate the required stack size or the Task Control Block (TCB). The task was not created.

Export to Sheets

6. Example

```
C
// ... task function vHighPriorityTask is defined ...

void app_main(void) {
    BaseType_t xReturned = xTaskCreatePinnedToCore(
        vHighPriorityTask,
        "HighPriTask",
        4096,
```

```
    NULL,  
    10,  
    NULL,  
    1           // Pin to Core 1  
);  
  
if (xReturned == pdPASS) {  
    printf("Task successfully pinned to Core 1.\n");  
} else {  
    // Handle memory failure!  
    printf("ERROR: Task creation failed, likely due to low memory.\n");  
}  
}
```

7. Workflow (What Happens Internally)

1. **Memory Allocation and Initialization:** TCB and Stack are allocated and initialized.
2. **Core Assignment:** FreeRTOS records the value of `xCoreID` in the task's TCB.
3. **State Change:** The task is placed into the **Ready List** specific to the assigned core.
4. **Scheduler Decision:** The scheduler on the specified core checks if this is the highest priority task and schedules it to run if possible, ensuring it **never** runs on the other core.

1. vTaskDelete()

2. Use Case (The "Why")

`vTaskDelete()` is used to **permanently remove a task** from the FreeRTOS system and free the memory (stack and Task Control Block - TCB) that the task was using.

- **Why Delete?** Tasks often run forever (using an infinite loop). However, sometimes a task is created to handle a temporary job (e.g., a one-time configuration process, or a connection that is about to close). Once that job is complete, the task is deleted to **reclaim its memory**, preventing memory leaks and freeing up resources for other uses.
-

3. Syntax

```
void vTaskDelete( TaskHandle_t xTaskToDelete );
```

4. Parameter

Parameter	Type	Description
<code>xTaskToDelete</code>	<code>TaskHandle_t</code>	The handle of the task you want to delete. This is the handle that was optionally returned by <code>xTaskCreate()</code> .

Export to Sheets

Special Case:

- If you pass **NULL** as the parameter, it instructs FreeRTOS to **delete the calling task itself** (the task that executes `vTaskDelete(NULL)`). This is the most common use case.

Return Value:

- The function returns **void** (nothing). If the function is called successfully, the task is removed, and the CPU immediately context-switches to the next ready task.
-

5. Example

Example A: Self-Deletion (Most Common)

A task completes its mission and deletes itself:

```
void vOneTimeTask(void *pvParameters) {
    printf("One-time task starting work...\n");

    // ... Code to perform a specific, non-repeating operation ...

    printf("One-time task finished. Deleting myself.\n");

    // Deletes the task that called this function
    vTaskDelete(NULL);

    // WARNING: Code below this line will NEVER be executed!
    // The task is already gone.
}
```

Example B: Deleting Another Task

A supervisor task monitors another task and deletes it when no longer needed:

TaskHandle_t xTargetTaskHandle = NULL; // Must be set during creation

```
void vSupervisorTask(void *pvParameters) {
    // Wait for the target task to finish its job (e.g., via a Queue/Event)
    vTaskDelay(pdMS_TO_TICKS(5000));

    if (xTargetTaskHandle != NULL) {
        printf("Supervisor deleting the target task.\n");
        // Deletes the task associated with xTargetTaskHandle
        vTaskDelete(xTargetTaskHandle);
        xTargetTaskHandle = NULL; // Good practice to null the handle
    }
}
```



```
// Supervisor continues its work...  
}
```

6. Workflow

1. **Deletion Call:** A task calls `vTaskDelete(xTaskToDelete)`.
2. **Task Removal:**
 - The task specified by `xTaskToDelete` is **removed from all RTOS lists** (Ready, Blocked, Suspended).
 - The task's priority is no longer considered by the scheduler.
3. **Memory Reclamation:** The memory used for the task's **Stack** and its **Task Control Block (TCB)** is returned to the heap. This makes the memory available for future use by other tasks or variables.
4. **Context Switch:** Since a task was just removed (or deleted itself), a **context switch is forced**. The scheduler finds the highest priority task currently in the Ready List and switches the CPU to that task.
5. **Task Termination:** The deleted task's code execution **stops permanently** at the point of the `vTaskDelete()` call. Its state is destroyed.

1. vTaskSuspend()

2. Use Case (The "Why")

`vTaskSuspend( )` is used to **temporarily take a task out of the scheduler's control**. A suspended task cannot execute, regardless of its priority, until it is explicitly resumed by another task using the `vTaskResume( )` function.

- **Why Suspend?** It's a method of controlling tasks that is **different from blocking** (like with `vTaskDelay( )`).
 - **Blocking** has a defined timeout period (e.g., wait 500ms).
 - **Suspension** is indefinite; the task is frozen until another task wakes it up.
 - This is often used for:
 - **Power Saving:** Suspending a task until an external event (like a button press or network activity) requires it.
 - **Debugging/System Control:** Pausing a problematic task or a task that is currently unnecessary to free up CPU cycles.
-

3. Syntax

```
void vTaskSuspend( TaskHandle_t xTaskToSuspend );
```

4. Parameter

Parameter	Type	Description
<code>xTaskToSuspend</code>	<code>TaskHandle_t</code>	The handle of the task you want to suspend.

Export to Sheets

Special Case:

- Like `vTaskDelete()`, if you pass **NULL** as the parameter, it instructs FreeRTOS to **suspend the calling task itself**.

Return Value:

- The function returns **void** (nothing).
-

5. Example

Example A: Suspending a Target Task

A supervisor task suspends a data logging task to perform a critical operation without interference:

```
TaskHandle_t xLoggerTaskHandle = NULL; // Must be set during creation
```

```
void vCriticalOpTask(void *pvParameters) {  
  
    // 1. Suspend the logger task  
    if (xLoggerTaskHandle != NULL) {  
        printf("Suspending logger task.\n");  
        vTaskSuspend(xLoggerTaskHandle);  
    }  
  
    // 2. Perform the critical operation that must not be interrupted  
    printf("Performing critical operation...\n");  
    // ... Critical code runs here ...  
    vTaskDelay(pdMS_TO_TICKS(100));  
  
    // 3. Resume the logger task  
    if (xLoggerTaskHandle != NULL) {  
        printf("Resuming logger task.\n");  
        vTaskResume(xLoggerTaskHandle);  
    }  
  
    // Task finishes or repeats...  
}
```

Example B: Self-Suspension

A task suspends itself, waiting to be woken up by an interrupt or event handler:

```
void vEventWaitTask(void *pvParameters) {
    for (;;) {
        printf("Event wait task ready. Suspending myself.\n");

        // Task stops here until vTaskResume() is called on its handle
        vTaskSuspend(NULL);

        // Execution resumes right here when vTaskResume() is called
        printf("Event received! Handling event...\n");
        // ... Handle the event ...
    }
}
```

6. Workflow (What Happens Internally)

1. **Suspension Call:** A task calls `vTaskSuspend(xTaskToSuspend)`.
2. **State Change:** FreeRTOS moves the target task (specified by the handle) from whichever list it was in (Ready, Blocked, or Running) to the **Suspended List**.
3. **Scheduler Exclusion:** Once in the Suspended List, the scheduler completely **ignores** the task. Its priority is irrelevant. The task is effectively frozen in its current state.
4. **Context Switch (if suspending itself):** If a task calls `vTaskSuspend(NULL)` (self-suspension), a context switch is immediately forced to allow the next highest priority task to run. The suspending task is removed from the CPU.
5. **Resumption:** The task remains in the Suspended List until **another task** calls `vTaskResume()` with the suspended task's handle. At that point, the task is moved back to the appropriate list (usually the Ready List) and can run again based on its priority.